

SaveTune - Documentación Completa del Proyecto

Índice

1. [Visión General](#)
 2. [Arquitectura del Sistema](#)
 3. [Backend \(Node.js\)](#)
 4. [Android App \(Kotlin\)](#)
 5. [Flujo de Datos](#)
 6. [Estructura de Archivos](#)
 7. [Plan de Desarrollo](#)
 8. [Despliegue](#)
-

1. Visión General

Objetivo

Aplicación Android que permite a los usuarios:

1. Pegar URL de playlist de Spotify
2. Ver todas las canciones de la playlist
3. Descargar cada canción como archivo MP3 desde YouTube

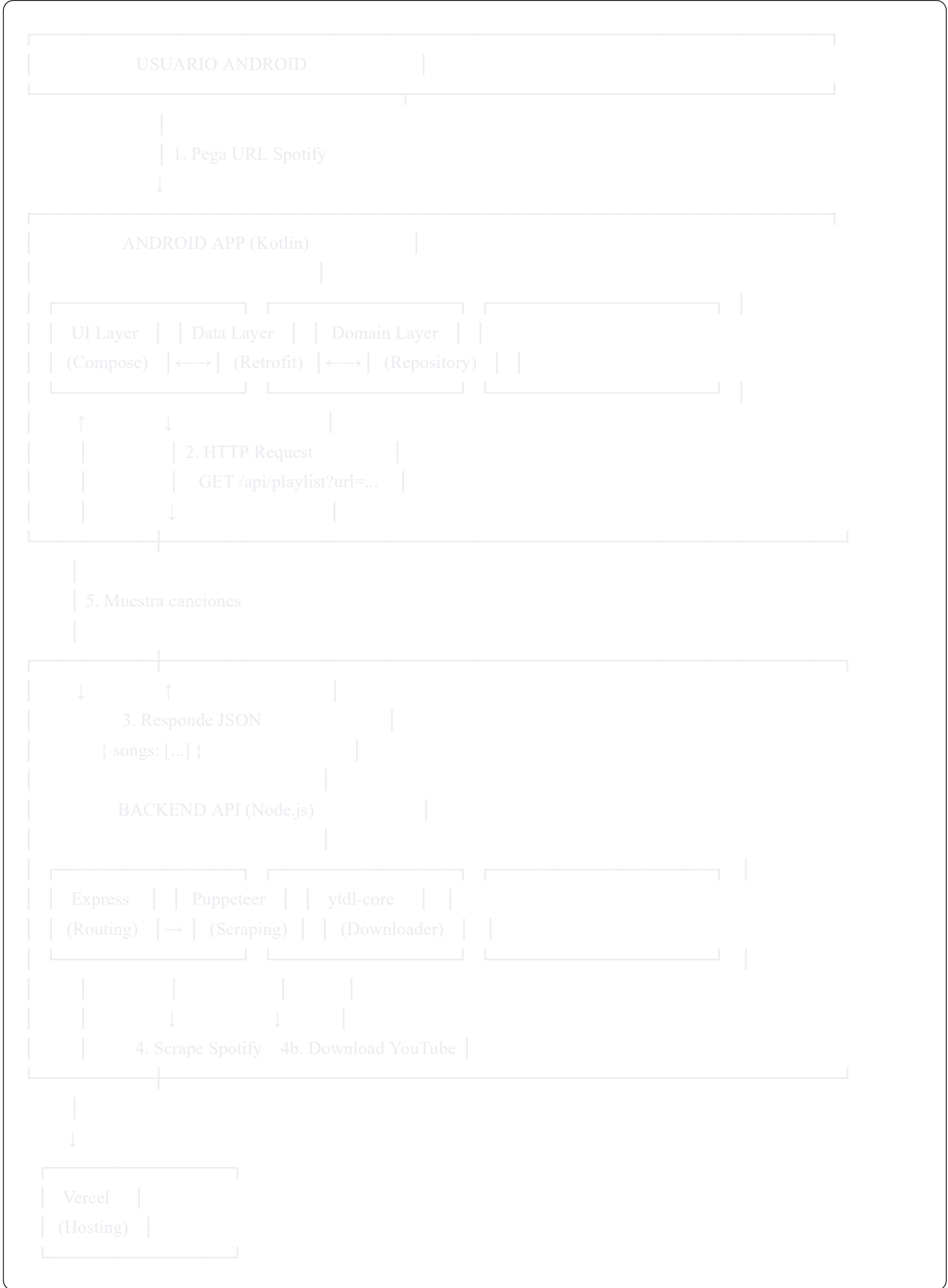
Stack Tecnológico

- **Backend:** Node.js + Express + Puppeteer + ytdl-core
- **Frontend:** Android (Kotlin) + Jetpack Compose + Retrofit
- **Despliegue:** Vercel (backend) + Google Play (app)

Motivación del Stack

- **Puppeteer en servidor:** Scraping robusto de Spotify con navegador real
 - **Kotlin nativo:** Mejor rendimiento y acceso a APIs de Android
 - **Backend separado:** Actualizaciones sin publicar nueva versión de app
-

2. Arquitectura del Sistema



3. Backend (Node.js)

3.1 Tecnologías

json

```
{
  "dependencies": {
    "express": "^4.18.2",
    "puppeteer": "^21.6.1",
    "ytdl-core": "^4.11.5",
    "cors": "^2.8.5",
    "helmet": "^7.1.0",
    "compression": "^1.7.4"
  }
}
```

3.2 Endpoints API

GET /api/playlist

Extrae canciones de una playlist de Spotify.

Request:

GET https://api.tudominio.com/api/playlist?url=https://open.spotify.com/playlist/37i9dQZF1DXcBWIGoYBM5M

Response:

json

```
{
  "success": true,
  "playlist": {
    "name": "Top 50 Global",
    "totalTracks": 50
  },
  "songs": [
    {
      "id": "1",
      "title": "As It Was",
      "artist": "Harry Styles",
      "imageUrl": "https://i.scdn.co/image/...",
      "duration": 167
    }
  ]
}
```

Errores:

json

```
{
  "success": false,
  "error": "Invalid Spotify URL",
  "message": "Please provide a valid Spotify playlist URL"
}
```

GET /api/search-youtube

Busca una canción en YouTube y devuelve el mejor resultado.

Request:

GET https://api.tudominio.com/api/search-youtube?query=Harry+Styles+As+It+Was

Response:

json

```
{
  "success": true,
  "video": {
    "id": "H5v3kku4y6Q",
    "title": "Harry Styles - As It Was (Official Video)",
    "url": "https://www.youtube.com/watch?v=H5v3kku4y6Q",
    "duration": "2:47",
    "thumbnail": "https://i.ytimg.com/vi/H5v3kku4y6Q/hqdefault.jpg"
  }
}
```

GET /api/download

Descarga audio de YouTube y devuelve el archivo.

Request:

GET https://api.tudominio.com/api/download?videoId=H5v3kku4y6Q

Response:

- Content-Type: `audio/mpeg`
- Stream de audio MP3

3.3 Estructura Backend

```
backend/
├── src/
│   ├── controllers/
│   │   ├── playlistController.js  # Lógica de Spotify scraping
│   │   └── downloadController.js  # Lógica de YouTube download
│   ├── services/
│   │   ├── spotifyService.js      # Puppeteer scraping
│   │   ├── youtubeService.js      # ytdl-core wrapper
│   │   └── searchService.js       # YouTube search
│   ├── utils/
│   │   ├── logger.js             # Winston logger
│   │   └── validators.js         # URL validation
│   ├── routes/
│   │   └── api.js                # Express routes
│   └── app.js                    # Express app setup
├── .env
├── package.json
├── vercel.json                  # Vercel config
└── README.md
```

3.4 Código Core del Backend

src/services/spotifyService.js

```
const puppeteer = require('puppeteer');
```

```
class SpotifyService {
  async getPlaylist(url) {
    let browser;

    try {
      browser = await puppeteer.launch({
        headless: 'new',
        args: ['--no-sandbox', '--disable-setuid-sandbox']
      });

      const page = await browser.newPage();

      // User agent para evitar detección
      await page.setUserAgent(
        'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
      );

      await page.goto(url, {
        waitUntil: 'networkidle2',
        timeout: 30000
      });

      // Esperar a que carguen las canciones
      await page.waitForSelector('a[href*="/track/"]', {
        timeout: 10000
      });

      // Extraer información de la playlist
      const playlistData = await page.evaluate(() => {
        // Nombre de la playlist
        const nameEl = document.querySelector('h1');
        const playlistName = nameEl ? nameEl.textContent.trim() : 'Unknown Playlist';

        // Canciones
        const songs = [];
        const seen = new Set();
        const trackLinks = document.querySelectorAll('a[href*="/track/"]');

        trackLinks.forEach(link => {
          const href = link.getAttribute('href');
          const trackId = href.split('/track/')[1]? split('?')[0];

          if (!trackId || seen.has(trackId)) return;
          seen.add(trackId);
        });

        // Buscar el contenedor de la canción
      });
    } catch (error) {
      console.error('Error al obtener la playlist:', error);
    }
  }
}
```

```
let row = link.closest('[data-testid="tracklist-row"]');
if (!row) row = link.closest('div[role="row"]');
if (!row) return;

// Título
const title = link.textContent.trim() || 'Unknown';

// Artistas
const artistLinks = row.querySelectorAll('a[href*="/artist/"]');
const artists = Array.from(artistLinks)
  .map(a => a.textContent.trim())
  .filter(Boolean)
  .join(', ') || 'Unknown Artist';

// Imagen
const img = row.querySelector('img');
const imageUrl = img ? img.src : "";

// Duración
let duration = 0;
const durationEl = row.querySelector('[data-testid="duration"]');
if (durationEl) {
  const timeStr = durationEl.textContent.trim();
  const parts = timeStr.split(':');
  if (parts.length === 2) {
    duration = parseInt(parts[0]) * 60 + parseInt(parts[1]);
  }
}

songs.push({
  id: trackId,
  title,
  artist: artists,
  imageUrl,
  duration
});
});

return {
  name: playlistName,
  songs
};
});

return {
  success: true,
  playlist: {
    name: playlistData.name,
    totalTracks: playlistData.songs.length
  },
}
```

```
    songs: playlistData.songs
  };

} catch (error) {
  console.error('Error scraping Spotify:', error);
  throw new Error('Failed to scrape Spotify playlist');
} finally {
  if (browser) await browser.close();
}
}

module.exports = new SpotifyService();
```

src/services/youtubeService.js

javascript

```
const ytdl = require('ytdl-core');
const { google } = require('googleapis');

class YouTubeService {
  constructor() {
    this.youtube = google.youtube({
      version: 'v3',
      auth: process.env.YOUTUBE_API_KEY // Opcional
    });
  }

  async searchVideo(query) {
    try {
      // Si tienes API key de YouTube
      if (process.env.YOUTUBE_API_KEY) {
        const response = await this.youtube.search.list({
          part: 'snippet',
          q: query,
          type: 'video',
          maxResults: 1,
          videoCategoryId: '10' // Música
        });

        const video = response.data.items[0];
        return {
          id: video.id.videoId,
          title: video.snippet.title,
          url: `https://www.youtube.com/watch?v=${video.id.videoId}`,
          thumbnail: video.snippet.thumbnails.high.url
        };
      } else {
        // Fallback: construir URL de búsqueda
        const searchUrl = `https://www.youtube.com/results?search_query=${encodeURIComponent(query)}`;
        // Aquí podrías usar Puppeteer para scrapear el primer resultado
        // Por simplicidad, devuelve la URL de búsqueda
        return {
          searchUrl,
          message: 'User must search manually'
        };
      }
    } catch (error) {
      console.error('Error searching YouTube:', error);
      throw new Error('Failed to search YouTube');
    }
  }

  async downloadAudio(videoUrl) {
  }
```

```

try {
  const info = await ytdl.getInfo(videoUrl);

  // Obtener el mejor formato de audio
  const audioFormats = ytdl.filterFormats(info.formats, 'audioonly');

  if (audioFormats.length === 0) {
    throw new Error('No audio formats available');
  }

  // Crear stream de descarga
  const stream = ytdl(videoUrl, {
    quality: 'highestaudio',
    filter: 'audioonly'
  });

  return {
    stream,
    title: info.videoDetails.title,
    duration: info.videoDetails.lengthSeconds
  };

} catch (error) {
  console.error('Error downloading from YouTube:', error);
  throw new Error('Failed to download audio');
}
}

module.exports = new YouTubeService();

```

src/app.js

javascript

```
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const compression = require('compression');
const apiRoutes = require('./routes/api');

const app = express();

// Middleware
app.use(helmet());
app.use(cors());
app.use(compression());
app.use(express.json());

// Routes
app.use('/api', apiRoutes);

// Health check
app.get('/health', (req, res) => {
  res.json({ status: 'ok', timestamp: new Date().toISOString() });
});

// Error handler
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({
    success: false,
    error: 'Internal Server Error',
    message: err.message
  });
});

module.exports = app;
```

src/routes/api.js

javascript

```
const express = require('express');
const router = express.Router();
const spotifyService = require('../services/spotifyService');
const youtubeService = require('../services/youtubeService');
```

```
// GET /api/playlist?url=...
```

```
router.get('/playlist', async (req, res, next) => {
  try {
    const { url } = req.query;

    if (!url || !url.includes('spotify.com/playlist/')) {
      return res.status(400).json({
        success: false,
        error: 'Invalid URL',
        message: 'Please provide a valid Spotify playlist URL'
      });
    }

    const result = await spotifyService.getPlaylist(url);
    res.json(result);

  } catch (error) {
    next(error);
  }
});
```

```
// GET /api/search-youtube?query=...
```

```
router.get('/search-youtube', async (req, res, next) => {
  try {
    const { query } = req.query;

    if (!query) {
      return res.status(400).json({
        success: false,
        error: 'Missing query',
        message: 'Please provide a search query'
      });
    }

    const result = await youtubeService.searchVideo(query);
    res.json({ success: true, video: result });

  } catch (error) {
    next(error);
  }
});
```

```
// GET /api/playlist?url=...
```

```
// GET /api/download?videoId=...
router.get('/download', async (req, res, next) => {
  try {
    const { videoId } = req.query;

    if (!videoId) {
      return res.status(400).json({
        success: false,
        error: 'Missing videoId',
        message: 'Please provide a YouTube video ID'
      });
    }

    const videoUrl = `https://www.youtube.com/watch?v=${videoId}`;
    const { stream, title } = await youtubeService.downloadAudio(videoUrl);

    // Configurar headers para descarga
    res.setHeader('Content-Type', 'audio/mpeg');
    res.setHeader('Content-Disposition', `attachment; filename="${title}.mp3"`);

    // Pipe el stream al response
    stream.pipe(res);

  } catch (error) {
    next(error);
  }
});

module.exports = router;
```

3.5 Variables de Entorno

```
env

# .env
NODE_ENV=production
PORT=3000
YOUTUBE_API_KEY=tu_api_key_opcional
```

3.6 Configuración Vercel

json

```
{
  "version": 2,
  "builds": [
    {
      "src": "src/app.js",
      "use": "@vercel/node"
    }
  ],
  "routes": [
    {
      "src": "/*",
      "dest": "src/app.js"
    }
  ]
}
```

4. Android App (Kotlin)

4.1 Tecnologías

kotlin

```
// build.gradle.kts (Project)
plugins {
    id("com.android.application") version "8.2.0" apply false
    id("org.jetbrains.kotlin.android") version "1.9.20" apply false
}

// build.gradle.kts (Module: app)
dependencies {
    // Compose
    val composeBom = platform("androidx.compose:compose-bom:2024.02.00")
    implementation(composeBom)
    implementation("androidx.compose.ui:ui")
    implementation("androidx.compose.material3:material3")
    implementation("androidx.compose.ui:ui-tooling-preview")
    implementation("androidx.activity:activity-compose:1.8.2")

    // Lifecycle
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.7.0")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.7.0")

    // Networking
    implementation("com.squareup.retrofit2:retrofit:2.9.0")
    implementation("com.squareup.retrofit2:converter-gson:2.9.0")
    implementation("com.squareup.okhttp3:okhttp:4.12.0")
    implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")

    // Coroutines
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")

    // Image loading
    implementation("io.coil-kt:coil-compose:2.5.0")

    // Navigation
    implementation("androidx.navigation:navigation-compose:2.7.6")
}
```

4.2 Estructura Android

```
app/
├── src/
│   └── main/
│       ├── java/com/tuapp/savetune/
│       │   ├── data/
│       │   │   ├── api/
│       │   │   │   ├── ApiService.kt
│       │   │   │   └── RetrofitClient.kt
│       │   │   └── models/
│       │   │       ├── Song.kt
│       │   │       ├── PlaylistResponse.kt
│       │   │       └── DownloadResponse.kt
│       │   └── repository/
│       │       └── SongRepository.kt
│       ├── ui/
│       │   ├── screens/
│       │   │   ├── HomeScreen.kt
│       │   │   └── PlaylistScreen.kt
│       │   ├── components/
│       │   │   ├── SongItem.kt
│       │   │   └── LoadingIndicator.kt
│       │   └── theme/
│       │       ├── Color.kt
│       │       ├── Theme.kt
│       │       └── Type.kt
│       ├── viewmodel/
│       │   └── PlaylistViewModel.kt
│       ├── utils/
│       │   ├── DownloadManager.kt
│       │   └── Constants.kt
│       └── MainActivity.kt
├── res/
│   ├── drawable/
│   ├── values/
│   │   ├── colors.xml
│   │   ├── strings.xml
│   │   └── themes.xml
│   └── xml/
│       └── network_security_config.xml
└── AndroidManifest.xml
build.gradle.kts
```

4.3 Modelos de Datos

data/models/Song.kt

kotlin

```
package com.tuapp.savetune.data.models

import com.google.gson.annotations.SerializedName

data class Song(
    @SerializedName("id")
    val id: String,

    @SerializedName("title")
    val title: String,

    @SerializedName("artist")
    val artist: String,

    @SerializedName("imageUrl")
    val imageUrl: String,

    @SerializedName("duration")
    val duration: Int = 0,

    // Estado local (no viene del servidor)
    var isDownloading: Boolean = false,
    var downloadProgress: Float = 0f,
    var isDownloaded: Boolean = false
)
```

data/models/PlaylistResponse.kt

kotlin

```
package com.tuapp.savetune.data.models

import com.google.gson.annotations.SerializedName

data class PlaylistResponse(
    @SerializedName("success")
    val success: Boolean,

    @SerializedName("playlist")
    val playlist: PlaylistInfo?,

    @SerializedName("songs")
    val songs: List<Song>,

    @SerializedName("error")
    val error: String? = null,

    @SerializedName("message")
    val message: String? = null
)

data class PlaylistInfo(
    @SerializedName("name")
    val name: String,

    @SerializedName("totalTracks")
    val totalTracks: Int
)
```

4.4 API Service

data/api/ApiService.kt

kotlin

```
package com.tuapp.savetune.data.api

import com.tuapp.savetune.data.models.PlaylistResponse
import okhttp3.ResponseBody
import retrofit2.Response
import retrofit2.http.GET
import retrofit2.http.Query
import retrofit2.http.Streaming

interface ApiService {

    @GET("playlist")
    suspend fun getPlaylist(
        @Query("url") url: String
    ): Response<PlaylistResponse>

    @GET("search-youtube")
    suspend fun searchYouTube(
        @Query("query") query: String
    ): Response<SearchResponse>

    @Streaming
    @GET("download")
    suspend fun downloadAudio(
        @Query("videoId") videoId: String
    ): Response<ResponseBody>
}

data class SearchResponse(
    val success: Boolean,
    val video: VideoInfo?
)

data class VideoInfo(
    val id: String,
    val title: String,
    val url: String,
    val thumbnail: String
)
```

data/api/RetrofitClient.kt

kotlin

```
package com.tuapp.savetune.data.api

import okhttp3.OkHttpClient
import okhttp3.logging.HttpLoggingInterceptor
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import java.util.concurrent.TimeUnit

object RetrofitClient {

    private const val BASE_URL = "https://tu-backend.vercel.app/api/"

    private val loggingInterceptor = HttpLoggingInterceptor().apply {
        level = HttpLoggingInterceptor.Level.BODY
    }

    private val okHttpClient = OkHttpClient.Builder()
        .addInterceptor(loggingInterceptor)
        .connectTimeout(30, TimeUnit.SECONDS)
        .readTimeout(60, TimeUnit.SECONDS)
        .writeTimeout(60, TimeUnit.SECONDS)
        .build()

    private val retrofit = Retrofit.Builder()
        .baseUrl(BASE_URL)
        .client(okHttpClient)
        .addConverterFactory(GsonConverterFactory.create())
        .build()

    val apiService: ApiService = retrofit.create(ApiService::class.java)
}
```

4.5 Repository

data/repository/SongRepository.kt

kotlin

```
package com.tuapp.savetune.data.repository

import com.tuapp.savetune.data.api.RetrofitClient
import com.tuapp.savetune.data.models.PlaylistResponse
import com.tuapp.savetune.data.models.Song
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import java.io.File
import java.io.FileOutputStream

class SongRepository {

    private val apiService = RetrofitClient.apiService

    suspend fun getPlaylistSongs(url: String): Result<PlaylistResponse> {
        return withContext(Dispatchers.IO) {
            try {
                val response = apiService.getPlaylist(url)

                if (response.isSuccessful && response.body() != null) {
                    Result.success(response.body()!!)
                } else {
                    Result.failure(Exception("Error: ${response.code()}"))
                }
            } catch (e: Exception) {
                Result.failure(e)
            }
        }
    }

    suspend fun searchAndDownloadSong(
        song: Song,
        outputFile: File,
        onProgress: (Float) -> Unit
    ): Result<File> {
        return withContext(Dispatchers.IO) {
            try {
                // 1. Buscar en YouTube
                val searchQuery = "${song.title} ${song.artist}"
                val searchResponse = apiService.searchYouTube(searchQuery)

                if (!searchResponse.isSuccessful || searchResponse.body()?.video == null) {
                    return@withContext Result.failure(
                        Exception("No se encontró la canción en YouTube")
                    )
                }
            }
        }
    }
}
```

```

val videoId = searchResponse.body()!!.video!!.id

// 2. Descargar audio
val downloadResponse = apiService.downloadAudio(videoId)

if (!downloadResponse.isSuccessful || downloadResponse.body() == null) {
    return@withContext Result.failure(
        Exception("Error descargando audio")
    )
}

// 3. Guardar archivo
val inputStream = downloadResponse.body()!!.byteStream()
val outputStream = FileOutputStream(outputFile)

val buffer = ByteArray(8192)
var totalBytes = 0L
val contentLength = downloadResponse.body()!!.contentLength()

inputStream.use { input ->
    outputStream.use { output ->
        var bytes = input.read(buffer)
        while (bytes >= 0) {
            output.write(buffer, 0, bytes)
            totalBytes += bytes

            // Reportar progreso
            if (contentLength > 0) {
                val progress = totalBytes.toFloat() / contentLength.toFloat()
                onProgress(progress)
            }

            bytes = input.read(buffer)
        }
    }
}

Result.success(outputFile)

} catch (e: Exception) {
    Result.failure(e)
}
}
}
}

```

4.6 ViewModel

viewmodel/PlaylistViewModel.kt

kotlin

```
package com.tuapp.savetune.viewmodel

import android.app.Application
import androidx.lifecycle.AndroidViewModel
import androidx.lifecycle.viewModelScope
import com.tuapp.savetune.data.models.PlaylistInfo
import com.tuapp.savetune.data.models.Song
import com.tuapp.savetune.data.repository.SongRepository
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch
import java.io.File

class PlaylistViewModel(application: Application) : AndroidViewModel(application) {

    private val repository = SongRepository()

    private val _uiState = MutableStateFlow(PlaylistUiState())
    val uiState: StateFlow<PlaylistUiState> = _uiState.asStateFlow()

    fun loadPlaylist(url: String) {
        viewModelScope.launch {
            _uiState.value = _uiState.value.copy(
                isLoading = true,
                error = null
            )

            repository.getPlaylistSongs(url)
                .onSuccess { response ->
                    _uiState.value = _uiState.value.copy(
                        isLoading = false,
                        playlistInfo = response.playlist,
                        songs = response.songs,
                        error = null
                    )
                }
                .onFailure { exception ->
                    _uiState.value = _uiState.value.copy(
                        isLoading = false,
                        error = exception.message ?: "Error desconocido"
                    )
                }
        }
    }

    fun downloadSong(song: Song) {
        viewModelScope.launch {
            repository.downloadSong(song)
        }
    }
}
```

```

viewModelScope.launch {
    // Actualizar estado: descargando
    updateSongState(song.id) {
        it.copy(isDownloading = true, downloadProgress = 0f)
    }

    val outputDir = File(
        getApplication<Application>().getExternalFilesDir(null),
        "SaveTune"
    )
    outputDir.mkdirs()

    val outputFile = File(outputDir, "${song.title} - ${song.artist}.mp3")

    repository.searchAndDownloadSong(
        song = song,
        outputFile = outputFile,
        onProgress = { progress ->
            updateSongState(song.id) {
                it.copy(downloadProgress = progress)
            }
        }
    )
    .onSuccess {
        updateSongState(song.id) {
            it.copy(
                isDownloading = false,
                isDownloaded = true,
                downloadProgress = 1f
            )
        }
    }
    .onFailure { exception ->
        updateSongState(song.id) {
            it.copy(isDownloading = false, downloadProgress = 0f)
        }
        _uiState.value = _uiState.value.copy(
            error = "Error: ${exception.message}"
        )
    }
}

private fun updateSongState(songId: String, update: (Song) -> Song) {
    val updatedSongs = _uiState.value.songs.map { song ->
        if (song.id == songId) update(song) else song
    }
    _uiState.value = _uiState.value.copy(songs = updatedSongs)
}

```



```
)  
  
data class PlaylistUiState(  
    val isLoading: Boolean = false,  
    val playlistInfo: PlaylistInfo? = null,  
    val songs: List<Song> = emptyList(),  
    val error: String? = null  
)
```

4.7 UI - Composables

ui/screens/HomeScreen.kt

kotlin

```
package com.tuapp.savetune.ui.screens
```

```
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.text.KeyboardActions
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Link
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.input.ImeAction
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import com.tuapp.savetune.viewmodel.PlaylistViewModel
```

@Composable

```
fun HomeScreen(
    viewModel: PlaylistViewModel = viewModel(),
    onNavigateToPlaylist: () -> Unit
) {
    var urlText by remember { mutableStateOf("") }
    val uiState by viewModel.uiState.collectAsState()

    // Navegar automáticamente cuando se cargue la playlist
    LaunchedEffect(uiState.songs) {
        if (uiState.songs.isNotEmpty()) {
            onNavigateToPlaylist()
        }
    }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text = "SAVETUNE",
            style = MaterialTheme.typography.displayLarge,
            color = MaterialTheme.colorScheme.primary
        )

        Spacer(modifier = Modifier.height(8.dp))

        Text(
            text = "Descubre la música que te gusta"
        )
    }
}
```

```
text = "Download playlists from Spotify",
style = MaterialTheme.typography.bodyLarge,
color = MaterialTheme.colorScheme.onSurfaceVariant
)
```

```
Spacer(modifier = Modifier.height(48.dp))
```

```
// URL Input
```

```
OutlinedTextField(
    value = urlText,
    onChange = { urlText = it },
    modifier = Modifier.fillMaxWidth(),
    label = { Text("Spotify Playlist URL") },
    leadingIcon = {
        Icon(Icons.Default.Link, contentDescription = null)
    },
    keyboardOptions = KeyboardOptions(
        imeAction = ImeAction.Done
    ),
    keyboardActions = KeyboardActions(
        onDone = {
            if (urlText.isNotBlank()) {
                viewModel.loadPlaylist(urlText)
            }
        }
    ),
    singleLine = true,
    enabled = !uiState.isLoading
)
```

```
Spacer(modifier = Modifier.height(16.dp))
```

```
// Analyze Button
```

```
Button(
    onClick = {
        if (urlText.isNotBlank()) {
            viewModel.loadPlaylist(urlText)
        }
    },
    modifier = Modifier
        .fillMaxWidth()
        .height(56.dp),
    enabled = !uiState.isLoading && urlText.isNotBlank()
) {
    if (uiState.isLoading) {
        CircularProgressIndicator(
            modifier = Modifier.size(24.dp),
            color = MaterialTheme.colorScheme.onPrimary
        )
    } else {
```

```

    ),
    Text("ANALYZE PLAYLIST")
  )
}

// Error message
if (uiState.error != null) {
  Spacer(modifier = Modifier.height(16.dp))
  Text(
    text = uiState.error!!,
    color = MaterialTheme.colorScheme.error,
    style = MaterialTheme.typography.bodyMedium
  )
}
}
}

```

ui/screens/PlaylistScreen.kt

kotlin

```
package com.tuapp.savetune.ui.screens
```

```
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import com.tuapp.savetune.ui.components.SongItem
import com.tuapp.savetune.viewmodel.PlaylistViewModel
```

```
@OptIn(ExperimentalMaterial3Api::class)
```

```
@Composable
```

```
fun PlaylistScreen(
    viewModel: PlaylistViewModel = viewModel(),
    onBackClick: () -> Unit
) {
    val uiState by viewModel.uiState.collectAsState()
```

```
Scaffold(
    topBar = {
        TopAppBar(
            title = {
                Column {
                    Text(
                        text = uiState.playlistInfo?.name ?: "Playlist",
                        style = MaterialTheme.typography.titleLarge
                    )
                    if (uiState.playlistInfo != null) {
                        Text(
                            text = "${uiState.playlistInfo!!.totalTracks} tracks",
                            style = MaterialTheme.typography.bodySmall
                        )
                    }
                }
            },
            navigationIcon = {
                IconButton(onClick = onBackClick) {
                    Icon(Icons.Default.ArrowBack, contentDescription = "Back")
                }
            }
        )
    }
)
```

```
)  
}  
) { padding ->  
    LazyColumn(  
        modifier = Modifier  
            .fillMaxSize()  
            .padding(padding),  
        contentPadding = PaddingValues(16.dp),  
        verticalArrangement = Arrangement.spacedBy(8.dp)  
    ) {  
        items(uiState.songs) { song ->  
            SongItem(  
                song = song,  
                onDownloadClick = { viewModel.downloadSong(song) }  
            )  
        }  
    }  
}  
}
```

ui/components/SongItem.kt

kotlin

```
package com.tuapp.savetune.ui.components
```

```
import androidx.compose.foundation.layout.*
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.CheckCircle
import androidx.compose.material.icons.filled.Download
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp
import coil.compose.AsyncImage
import com.tuapp.savetune.data.models.Song
```

```
@Composable
```

```
fun SongItem(
    song: Song,
    onDownloadClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Card(
        modifier = modifier.fillMaxWidth()
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(12.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            // Album art
            AsyncImage(
                model = song.imageUrl,
                contentDescription = song.title,
                modifier = Modifier.size(60.dp),
                contentScale = ContentScale.Crop
            )

            Spacer(modifier = Modifier.width(12.dp))

            // Song info
            Column(
                modifier = Modifier.weight(1f)
            ) {
                Text(
                    text = song.title,
                    style = MaterialTheme.typography.titleMedium,
                    color = MaterialTheme.colors.onSurface,
                    modifier = Modifier.padding(0 12.dp 0 0)
                )
                Text(
                    text = song.artist,
                    style = MaterialTheme.typography.titleMedium,
                    color = MaterialTheme.colors.onSurface,
                    modifier = Modifier.padding(0 12.dp 0 0)
                )
                Text(
                    text = song.album,
                    style = MaterialTheme.typography.titleMedium,
                    color = MaterialTheme.colors.onSurface,
                    modifier = Modifier.padding(0 12.dp 0 0)
                )
                Text(
                    text = song.duration,
                    style = MaterialTheme.typography.titleMedium,
                    color = MaterialTheme.colors.onSurface,
                    modifier = Modifier.padding(0 12.dp 0 0)
                )
            }
        }
    }
}
```

```

        style = MaterialTheme.typography.titleMedium,
        maxLines = 1,
        overflow = TextOverflow.Ellipsis
    )
    Text(
        text = song.artist,
        style = MaterialTheme.typography.bodyMedium,
        color = MaterialTheme.colorScheme.onSurfaceVariant,
        maxLines = 1,
        overflow = TextOverflow.Ellipsis
    )
    if (song.duration > 0) {
        val minutes = song.duration / 60
        val seconds = song.duration % 60
        Text(
            text = String.format("%d:%02d", minutes, seconds),
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
}

```

```

Spacer(modifier = Modifier.width(12.dp))

```

```

// Download button/status

```

```

when {
    song.isDownloading -> {
        Box(
            modifier = Modifier.size(48.dp),
            contentAlignment = Alignment.Center
        ) {
            CircularProgressIndicator(
                progress = song.downloadProgress,
                modifier = Modifier.size(40.dp)
            )
            Text(
                text = "${(song.downloadProgress * 100).toInt()}%",
                style = MaterialTheme.typography.labelSmall
            )
        }
    }
    song.isDownloaded -> {
        Icon(
            imageVector = Icons.Default.CheckCircle,
            contentDescription = "Downloaded",
            modifier = Modifier.size(40.dp),
            tint = MaterialTheme.colorScheme.primary
        )
    }
    else -> {

```



```
(  
    IconButton(  
        onClick = onDownloadClick,  
        modifier = Modifier.size(48.dp)  
    ) {  
        Icon(  
            imageVector = Icons.Default.Download,  
            contentDescription = "Download",  
            modifier = Modifier.size(32.dp)  
        )  
    }  
}  
}  
}  
}  
}
```

4.8 MainActivity

MainActivity.kt

kotlin

```
package com.tuapp.savetune

import android.Manifest
import android.content.pm.PackageManager
import android.os.Build
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.core.content.ContextCompat
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import com.tuapp.savetune.ui.screens.HomeScreen
import com.tuapp.savetune.ui.screens.PlaylistScreen
import com.tuapp.savetune.ui.theme.SaveTuneTheme
import com.tuapp.savetune.viewmodel.PlaylistViewModel

class MainActivity : ComponentActivity() {

    private val requestPermissionLauncher = registerForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { isGranted ->
        // Manejar resultado del permiso
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // Solicitar permisos de almacenamiento
        checkStoragePermission()

        setContent {
            SaveTuneTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    SaveTuneApp()
                }
            }
        }
    }
}
```


4.9 AndroidManifest.xml

xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <!-- Permisos -->
    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
    <uses-permission
        android:name="android.permission.WRITE_EXTERNAL_STORAGE"
        android:maxSdkVersion="32" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.SaveTune"
        android:networkSecurityConfig="@xml/network_security_config"
        android:usesCleartextTraffic="false">

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:theme="@style/Theme.SaveTune">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

5. Flujo de Datos

Secuencia completa de una descarga

1. Usuario pega URL en HomeScreen
- ↓
2. PlaylistViewModel.loadPlaylist(url)
- ↓
3. SongRepository.getPlaylistSongs(url)
- ↓
4. HTTP GET → Backend /api/playlist?url=...
- ↓
5. Backend: Puppeteer scrape Spotify
- ↓
6. Backend responde JSON con canciones
- ↓
7. UI muestra lista en PlaylistScreen
- ↓
8. Usuario pulsa botón descargar
- ↓
9. PlaylistViewModel.downloadSong(song)
- ↓
10. SongRepository.searchAndDownloadSong()
 - ├─ HTTP GET → /api/search-youtube?query=...
 - ├─ Backend busca en YouTube
 - ├─ HTTP GET → /api/download?videoId=...
 - └─ Backend stream MP3
- ↓
11. App guarda archivo local
- ↓
12. UI muestra ✓ completado

6. Plan de Desarrollo

Fase 1: Backend MVP (Semana 1)

Día 1-2: Setup y estructura

- ☐ Crear proyecto Node.js
- ☐ Instalar dependencias (express, puppeteer, ytdl-core)
- ☐ Configurar estructura de carpetas
- ☐ Crear endpoint básico (/health)

Día 3-4: Spotify scraping

- ☐ Implementar SpotifyService con Puppeteer
- ☐ Crear endpoint (/api/playlist)
- ☐ Probar con playlists públicas
- ☐ Manejar errores y timeouts

Día 5-6: YouTube integration

- ☐ Implementar YouTubeService
- ☐ Crear endpoint (/api/search-youtube)
- ☐ Crear endpoint (/api/download)
- ☐ Probar descarga completa

Día 7: Testing y deploy

- ☐ Crear tests con Postman
- ☐ Configurar Vercel
- ☐ Desplegar a producción
- ☐ Documentar API

Fase 2: Android App (Semana 2-3)

Día 1-2: Setup proyecto

- ☐ Crear proyecto Android en Android Studio
- ☐ Configurar Compose y dependencias
- ☐ Crear estructura de paquetes
- ☐ Configurar Retrofit

Día 3-4: Modelos y Repository

- ☐ Crear data models
- ☐ Implementar ApiService
- ☐ Implementar SongRepository
- ☐ Crear ViewModel básico

Día 5-6: UI - HomeScreen

- ☐ Implementar HomeScreen con Compose
- ☐ Input de URL
- ☐ Botón analizar
- ☐ Loading y error states

Día 7-8: UI - PlaylistScreen

- ☐ Implementar PlaylistScreen
- ☐ Lista de canciones con LazyColumn
- ☐ SongItem component
- ☐ Integrar con ViewModel

Día 9-10: Funcionalidad descarga

- ☐ Implementar lógica de descarga
- ☐ Progress tracking
- ☐ Guardar archivos localmente
- ☐ Permisos de almacenamiento

Día 11-12: Testing

- ☐ Testing en emulador
- ☐ Testing en dispositivo real
- ☐ Fix bugs encontrados
- ☐ Optimizar UX

Día 13-14: Polish

- ☐ Mejorar UI/UX
- ☐ Agregar animaciones
- ☐ Iconos y assets
- ☐ Preparar para release

Fase 3: Mejoras (Semana 4+)

- ☐ Cache local con Room
 - ☐ Búsqueda de canciones
 - ☐ Player integrado
 - ☐ Gestión de playlists
 - ☐ Tema oscuro/claro
 - ☐ Notificaciones de descarga
 - ☐ Widget
 - ☐ Compartir canciones
-

7. Despliegue

Backend (Vercel)

```
bash

# 1. Instalar Vercel CLI
npm install -g vercel

# 2. Login
vercel login

# 3. Deploy
cd backend
vercel

# 4. Configurar dominio (opcional)
vercel domains add tudominio.com
```

Android App

Build APK:

```
bash  
  
cd android  
./gradlew assembleRelease
```

Ubicación: `app/build/outputs/apk/release/app-release.apk`

Firmar APK:

```
bash  
  
# Generar keystore  
keytool -genkey -v -keystore savetune.keystore -alias savetune -keyalg RSA -keysize 2048 -validity 10000  
  
# Firmar  
jarsigner -verbose -sigalg SHA256withRSA -digestalg SHA-256 -keystore savetune.keystore app-release.apk savetune
```

Publicar en Google Play:

1. Crear cuenta de desarrollador (\$25 única vez)
 2. Crear aplicación en Play Console
 3. Subir APK firmado
 4. Completar ficha de la tienda
 5. Enviar a revisión
-

8. Consideraciones Importantes

Backend

Límites de Vercel Free Tier:

- 100GB bandwidth/mes
- 100 horas serverless/mes
- 10 segundos timeout por request

Si necesitas más:

- Railway: 500 horas gratis/mes
- Render: 750 horas gratis/mes
- Fly.io: 3 VMs pequeñas gratis

Puppeteer en Vercel:

- Usa `puppeteer-core` + `chrome-aws-lambda`
- Configuración especial en `vercel.json`

Android

Almacenamiento:

- Android 13+: App-specific storage (no permisos)
- Android 12-: WRITE_EXTERNAL_STORAGE
- Considera usar MediaStore API

Network Security:

- HTTPS obligatorio en producción
- Configura `network_security_config.xml`

Obfuscación:

- Habilita ProGuard/R8 para release
 - Protege código y reduce tamaño
-

9. Notas Finales

Alternativas a considerar

Si Puppeteer es muy pesado:

- Usar Spotify Web API oficial (requiere OAuth)
- Scraping ligero con Cheerio (sin navegador)

Si YouTube bloquea descargas:

- Usar YouTube Data API v3 (1000 requests gratis/día)
- Considerar servicios de terceros (RapidAPI)

Para iOS en el futuro:

- Kotlin Multiplatform para compartir código
- O Swift nativo consumiendo el mismo backend

Recursos útiles

- **Puppeteer Docs:** <https://pptr.dev/>
 - **Retrofit Guide:** <https://square.github.io/retrofit/>
 - **Compose Docs:** <https://developer.android.com/jetpack/compose>
 - **Vercel Docs:** <https://vercel.com/docs>
-

10. Checklist de Inicio

Para el agente que va a desarrollar:

Backend

- ☐ Copiar estructura de carpetas
- ☐ Instalar Node.js 18+
- ☐ Crear `package.json` con dependencias
- ☐ Implementar `spotifyService.js`
- ☐ Implementar `youtubeService.js`
- ☐ Crear routes en `api.js`
- ☐ Configurar `vercel.json`
- ☐ Probar con Postman
- ☐ Desplegar en Vercel

Android

- ☐ Instalar Android Studio
- ☐ Crear proyecto con Kotlin + Compose
- ☐ Configurar `build.gradle.kts`
- ☐ Copiar estructura de paquetes
- ☐ Implementar modelos de datos
- ☐ Implementar ApiService y Repository
- ☐ Crear ViewModel
- ☐ Implementar HomeScreen
- ☐ Implementar PlaylistScreen
- ☐ Configurar permisos en Manifest
- ☐ Probar en emulador
- ☐ Build APK de prueba

¡Listo para empezar! 🚀